VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Student Name	Registration Number
Arjun Tiwari	24BAI10141
Slot	**Date of Submission**
E-22	30 March 2026

1. Introduction

Git is a version control system that helps developers keep track of changes made to their code over time. It was created by Linus Torvalds in 2005 and is now maintained by Junio Hamano. Git is used by millions of developers worldwide, and its popularity can be seen in the fact that most open source projects use it. One of the key features of Git is its ability to allow multiple developers to collaborate on a project simultaneously. It does this by creating a new branch for each developer, which can then be merged back into the main branch once the changes have been made. This makes it easy for developers to work on different parts of a project without interfering with each other. Git also has a large community of users, which means there are many resources available to help with any issues that may arise. There are many different ways to use Git, including through the command line, or through a graphical user interface such as GitHub Desktop. Git is also highly customizable, with many different options available to suit the needs of different users. Overall, Git is a powerful tool that has revolutionized the way developers work on projects. It is free to use and is available for Windows, macOS, and Linux. Git has become an essential tool for any developer, and its use is now expected in most development environments. With its ease of use, flexibility, and large community, it's no wonder that Git has become the go-to version control system for so many developers. Git's impact on the development community has been huge, and it continues to be a vital part of the way we work on projects today.

Part A — Origin and Philosophy

A1. Problem this software solves

Before Git, developers had to rely on other version control systems, such as Subversion or CVS. These systems had many limitations, including being slow and cumbersome to use. They also didn't allow for the same level of collaboration that Git does, making it difficult for multiple developers to work on a project at the same time. Git solves this problem by providing a fast, flexible, and powerful version

control system that can handle large projects with many collaborators. It also provides a high level of security, with features such as encryption and access controls. One of the main problems that Git solves is the issue of version conflicts. When multiple developers are working on a project, it's easy for conflicts to arise, where two or more developers have made changes to the same code. Git solves this problem by providing a system of branching and merging, which allows developers to work on different versions of the code without interfering with each other. Git also provides a system of commit history, which allows developers to see exactly what changes have been made to the code over time. This makes it easy to track down bugs and fix problems. Overall, Git provides a comprehensive solution to the problems of version control, and has become the standard tool for developers worldwide. Git's ability to handle large projects and multiple collaborators has made it an essential tool for many companies and organizations. Its flexibility and customizability have also made it a popular choice for developers, who can tailor it to suit their specific needs. The problem that Git solves is a complex one, and it has taken many years of development to get to where it is today.

A2. License Analysis

Git is released under the GNU General Public License, which is a free software license that allows users to run, study, share, and modify the software. This license is widely used in the open source community, and is seen as a standard for open source software. The GPL license requires that any modifications to the software must also be released under the same license, which helps to ensure that the software remains free and open. This license has been chosen for Git because it provides a high level of freedom for users, while also protecting the rights of the original developers. The GPL license also provides a level of protection for users, as it ensures that the software will always remain free and open. One of the key benefits of the GPL license is that it allows for a high level of collaboration and community involvement. Because the software is open source, developers from all

over the world can contribute to it, which helps to drive innovation and improvement. The GPL license also provides a level of transparency, as all changes to the software must be made publicly available. This helps to build trust with users, who can see exactly what changes have been made to the software. Overall, the GPL license is a key part of what makes Git so successful, as it provides a framework for collaboration and community involvement. The GPL license has been widely adopted in the open source community, and is seen as a standard for open source software. Its use in Git has helped to drive the software's success, and has provided a high level of freedom and protection for users. The GPL license is an essential part of the open source ecosystem, and its use in Git has helped to promote collaboration and community involvement.

A3. Ethics of Open Source

The ethics and community values behind Git are centered around the idea of collaboration and community involvement. The software is designed to be open and transparent, with all changes and decisions made publicly available. This helps to build trust with users, who can see exactly what is going on with the software. The Git community is also highly values-driven, with a strong emphasis on freedom, openness, and collaboration. The community is made up of developers from all over the world, who work together to drive innovation and improvement. One of the key values of the Git community is the idea of meritocracy, where decisions are made based on the merit of the idea, rather than the status of the person proposing it. This helps to ensure that the best ideas rise to the top, and that the software is always improving. The Git community also places a strong emphasis on inclusivity and diversity, with a goal of making the software accessible to as many people as possible. This includes providing translations and documentation in many different languages, as well as making the software available on a wide range of platforms. Overall, the ethics and community values behind Git are centered around the idea of building a strong, collaborative community that is driven by a shared set of values. The Git

community is highly active, with many different channels and forums available for discussion and collaboration. The community is also highly supportive, with many different resources available to help new users get started. The values of the Git community are an essential part of what makes the software so successful, and have helped to drive its widespread adoption.

Part B — Linux Footprint

Git has a significant footprint on Linux, with many different distributions and packages available. The software is widely used in the Linux community, and is seen as an essential tool for many developers. Git is also highly customizable, with many different options available to suit the needs of different users. One of the key benefits of using Git on Linux is the high level of integration with other Linux tools and software. For example, Git can be easily integrated with other version control systems, such as Subversion, and can also be used with a wide range of text editors and IDEs. Git is also widely used in the Linux kernel development process, where it is used to manage the large and complex codebase. The use of Git in Linux kernel development has helped to drive innovation and improvement, and has provided a high level of transparency and accountability. Overall, Git has a significant impact on the Linux ecosystem, and is seen as an essential tool for many developers. The use of Git on Linux has also helped to drive the adoption of other open source software, and has provided a high level of collaboration and community involvement. The Linux community is highly supportive of Git, and many different resources are available to help new users get started. The integration of Git with other Linux tools and software has also helped to drive its widespread adoption, and has made it an essential part of the Linux ecosystem.

Part C — FOSS Ecosystem

The FOSS ecosystem around Git is highly active and diverse, with many different projects and communities available. One of the key

benefits of the FOSS ecosystem is the high level of collaboration and community involvement, with many different developers and users working together to drive innovation and improvement. The FOSS ecosystem also provides a high level of transparency and accountability, with all changes and decisions made publicly available. Git is also widely used in the FOSS ecosystem, with many different projects and communities relying on it as a key tool. The use of Git in the FOSS ecosystem has helped to drive the adoption of other open source software, and has provided a high level of collaboration and community involvement. One of the key benefits of the FOSS ecosystem is the high level of freedom and flexibility that it provides, with users and developers free to modify and customize the software to suit their needs. The FOSS ecosystem also provides a high level of security, with many different projects and communities working together to identify and fix vulnerabilities. Overall, the FOSS ecosystem around Git is highly active and diverse, and provides a high level of collaboration, transparency, and accountability. The use of Git in the FOSS ecosystem has helped to drive innovation and improvement, and has provided a high level of freedom and flexibility for users and developers. The FOSS ecosystem is an essential part of the open source community, and has helped to drive the widespread adoption of open source software.

Part D — Open Source vs Proprietary

Git is often compared to proprietary alternatives, such as Microsoft Team Foundation Server or Perforce. One of the key benefits of Git is its high level of flexibility and customizability, which allows it to be tailored to suit the needs of different users and projects. Proprietary alternatives often lack this level of flexibility, and can be more difficult to use and integrate with other tools and software. Git is also highly scalable, and can handle large and complex projects with ease. Proprietary alternatives often struggle with scalability, and can become slow and cumbersome as the project size increases. Another key benefit of Git is its high level of security, with features such as encryption and access controls. Proprietary alternatives often lack

these features, or may charge extra for them. Overall, Git is a highly capable and flexible version control system that is well-suited to a wide range of projects and users. Its high level of scalability, security, and customizability make it a popular choice for many developers, and its widespread adoption is a testament to its effectiveness. The use of Git has also helped to drive the adoption of other open source software, and has provided a high level of collaboration and community involvement. The comparison between Git and proprietary alternatives is often favorable, with Git emerging as a highly capable and flexible solution that is well-suited to a wide range of projects and users.

Shell Scripts and Outputs

Script 1: 01-identify.sh

```
#!/bin/bash
#                   Author:                   Arjun                   (24BAI10141)
#  ---------------------------------------------------------------------------
#                              Script:                              01-identify.sh
#   Purpose:   Gather   basic   system   information   for   the   Git   audit.
#  It extracts the Linux distribution, kernel version, current user, home
directory,
#  system uptime, and the current date/time. Finally, it prints a hard-coded
#              Open              Source              freedom              message.
#  ---------------------------------------------------------------------------

#  Function  to  safely  retrieve  the  Linux  distribution  name  and  version.
get_distro()                                                                    {
  if          [[          -f          /etc/os-release          ]];          then
    #     Source     the     file     to     get     PRETTY_NAME     variable.
    .                                                              /etc/os-release
    echo                                                            "$PRETTY_NAME"
  elif      command      -v      lsb_release      >/dev/null      2>&1;      then
    lsb_release                                                              -ds
  else
    echo                          "Unknown                          Distribution"
  fi
}

#        Retrieve        information        using        standard        commands.
DISTRO=$(get_distro)                                  # e.g., Ubuntu 22.04.3 LTS
KERNEL=$(uname  -r)                                            #  Kernel  version
CURRENT_USER=$(whoami)                                  # Current logged-in user
HOME_DIR=$HOME                                             # Home directory path
UPTIME=$(uptime -p)                                # Human readable uptime (e.g.,
up                2                hours,                5                minutes)
CURRENT_DATE=$(date     "%a     %b     %d     %Y     %H:%M:%S     %Z%z     (%Z)")

#           Print           a           nicely           formatted           report.
printf
"==========================================================================
==\n"
printf "                    Git AUDIT - SYSTEM IDENTITY                    \n"
printf
"==========================================================================
==\n"
printf          "Linux          Distribution:          %s\n"          "$DISTRO"
printf      "Kernel      Version:                          %s\n"      "$KERNEL"
printf    "Current    User:                          %s\n"    "$CURRENT_USER"
printf      "Home      Directory:                          %s\n"      "$HOME_DIR"
printf      "System      Uptime:                          %s\n"      "$UPTIME"
printf        "Current        Date/Time:              %s\n"        "$CURRENT_DATE"
printf  "--------------------------------------------------------------------------
```

```
---------\n"
printf "Message: This system runs on Open Source software, providing freedom to
study,                 change,                 and                 distribute.\n"
printf
"==============================================================================
==\n"
```

This script is pretty straightforward - it gathers basic system info and prints it out in a neat format. It starts by figuring out what Linux distribution you're running, like Ubuntu or something, and then gets the kernel version, the current user, home directory, system uptime, and the current date and time. It grabs all this info using standard commands like `uname` and `date`, and then prints it out in a nicely formatted report, including a message about the system running on open source software. When you run the script, you should see a table-like output in your terminal with all this system info, making it easy to quickly identify key details about your system at a glance.

Script 2: 02-packages.sh

```
#!/bin/bash
#                   Author:                   Arjun                   (24BAI10141)
#  ---------------------------------------------------------------------------
#                              Script:                              02-packages.sh
#  Purpose:  Detect  the  package  manager,  verify  Git  installation,  report  its
version,
#   and   display   philosophy   notes   for   four   popular   FOSS   tools.
#  ---------------------------------------------------------------------------

#   Detect   the   package   manager   and   set   a   variable   accordingly.
detect_pkg_manager()                                                                {
  if       command       -v       apt-get       >/dev/null       2>&1;       then
    echo                                                                      "apt"
  elif       command       -v       dnf       >/dev/null       2>&1;       then
    echo                                                                      "dnf"
  elif       command       -v       yum       >/dev/null       2>&1;       then
    echo                                                                      "yum"
  elif       command       -v       pacman       >/dev/null       2>&1;       then
    echo                                                                  "pacman"
  else
    echo                                                                 "unknown"
  fi
}

PKG_MANAGER=$(detect_pkg_manager)
PACKAGE_NAME="git"

#   Function   to   check   installation   status   and   retrieve   version.
check_git()                                                                         {
  case                              "$PKG_MANAGER"                              in
    apt)
      if     dpkg     -s     "$PACKAGE_NAME"     >/dev/null     2>&1;     then
        INSTALLED="YES"
        VERSION=$(dpkg  -s  "$PACKAGE_NAME"  |  awk  -F  ':  '  '/^Version/  {print
$2}')
      else
        INSTALLED="NO"
      fi
      ;;
    dnf|yum)
      if     rpm     -q     "$PACKAGE_NAME"     >/dev/null     2>&1;     then
        INSTALLED="YES"
        VERSION=$(rpm      -q      --queryformat      "%{VERSION}-%{RELEASE}\n"
"$PACKAGE_NAME")
      else
        INSTALLED="NO"
      fi
      ;;
    pacman)
      if     pacman     -Qi     "$PACKAGE_NAME"     >/dev/null     2>&1;     then
        INSTALLED="YES"
```

```
        VERSION=$(pacman -Qi "$PACKAGE_NAME" | awk -F ': ' '/^Version/ {print
$2}')
      else
        INSTALLED="NO"
      fi
      ;;
    *)
      INSTALLED="UNKNOWN"
      ;;
  esac
}

check_git

#  ------------------------------------------------------------------------------
#                                  Output                                  Section
#  ------------------------------------------------------------------------------
printf
"================================================================================
==\n"
printf "                     Git AUDIT - PACKAGE INSPECTOR                  \n"
printf
"================================================================================
==\n"
if        [[        "$INSTALLED"        ==        "YES"        ]];        then
  printf "Status: %s is INSTALLED on this %s system.\n" "$PACKAGE_NAME" "$(awk
-F=     '/^ID=/{print     $2}'     /etc/os-release     |     tr     -d     '"')"
  printf                "Version:                %s\n"                "$VERSION"
else
  printf  "Status:  %s  is  NOT  installed  on  this  system.\n"  "$PACKAGE_NAME"
fi
printf  "------------------------------------------------------------------------
---------\n"
printf                "FOSS                Philosophy                Notes:\n"
case                            "$PACKAGE_NAME"                            in
  git)
    printf  "  -  Git:  A  distributed  version-control  system  that  embodies  the
freedom            to            collaborate            and            fork.\n"
    ;;
esac
#       Additional       notes       for       three       other       tools.
printf " - Vim: A powerful, extensible editor that respects the user's control
over                their                editing                environment.\n"
printf " - GCC: The GNU Compiler Collection, enabling developers to build free
software                        across                        architectures.\n"
printf " - Python: An open, community-driven language that encourages readable
and                             shareable                             code.\n"
printf
"================================================================================
==\n"
```

This script checks if Git is installed on your system and reports its version. It first detects the package manager you're using, like apt or

dnf, and then uses that to check if Git is installed. If Git is installed, it shows the status and version, and if not, it just shows that it's not installed. The script also prints out some notes about the philosophy behind Git and a few other popular open-source tools like Vim, GCC, and Python. When you run the script, you should see a formatted output that includes the installation status and version of Git, along with these extra notes about the other tools.

Script 3: 03-auditor.sh

```
#!/bin/bash
#                    Author:                    Arjun                    (24BAI10141)
#  ---------------------------------------------------------------------------
#                               Script:                               03-auditor.sh
# Purpose: Audit a set of critical directories (including Git-specific ones) and
#  report  their  size,  permissions,  and  owner  in  a  clean  columnar  format.
#  ---------------------------------------------------------------------------

#  Base  directories  that  are  generally  important  for  system  audits.
DIRS=(
  "/etc"
  "/var/log"
)

#  Dynamically  append  Git-specific  paths.  These  may  vary  by  distribution.
DIRS+=("/usr/share/git-core"                                    "/etc/gitconfig")

#           Header           for           the           output           table.
printf  "%-30s  %-10s  %-10s  %-20s\n"  "Directory"  "Size"  "Perms"  "Owner"
printf "%s\n" "-----------------------------------------------------------------
----------------"

#  Loop  through  each  directory,  verify  existence,  and  extract  details.
for              DIR              in              "${DIRS[@]}";              do
  if            [[            -d            "$DIR"            ]];            then
    # Use du to get the total size in a human-readable format (summarize the
dir).
    SIZE=$(du      -sh      "$DIR"      2>/dev/null      |      cut      -f1)
    #  Use  stat  to  fetch  permissions  (in  octal)  and  owner  name.
    PERMS=$(stat                -c                "%A"                "$DIR")
    OWNER=$(stat                -c                "%U"                "$DIR")
    printf  "%-30s  %-10s  %-10s  %-20s\n"  "$DIR"  "$SIZE"  "$PERMS"  "$OWNER"
  else
    #   If   the   directory   does   not   exist,   note   it   clearly.
    printf  "%-30s  %-10s  %-10s  %-20s\n"  "$DIR"  "N/A"  "N/A"  "N/A  (missing)"
  fi
done
```

This script audits a set of important directories on the system, including some Git-specific ones, and reports back their size, permissions, and owner. It first checks if each directory exists, and if it does, it uses the `du` and `stat` commands to get the size and permissions/owner info. The script then prints out this info in a neat table format, making it easy to read and compare the different directories. If a directory doesn't exist, it notes that in the output

instead. So, when you run this script, you should see a clean table in the terminal with columns for the directory name, size, permissions, and owner, giving you a quick overview of these critical system directories.

Script 4: 04-logs.sh

```
#!/bin/bash
#                   Author:                   Arjun                   (24BAI10141)
#  ---------------------------------------------------------------------------
#                               Script:                               04-logs.sh
# Purpose: Analyze a given log file for a specific keyword, count occurrences,
#      and      display      the      last      five      matching      lines.
#      Usage:                   ./04-logs.sh      /path/to/logfile      keyword
#  ---------------------------------------------------------------------------

#    Ensure    a    log    file    path    and    a    keyword    are    supplied.
if       [[       -z       "$1"       ||       -z       "$2"       ]];       then
  echo          "Usage:          $0          <log_file_path>          <keyword>"
  exit                                                                          1
fi

LOG_FILE="$1"
KEYWORD="$2"

#  Suggest  a  realistic  Git  log  location  for  reference  (comment  only).
# Example Git log file: /var/log/git.log or /var/log/syslog (where Git daemon
writes).

#    Verify    that    the    log    file    exists    and    is    readable.
if         [[         !         -r         "$LOG_FILE"         ]];         then
  echo      "Error:      Cannot      read      log      file      '$LOG_FILE'."
  exit                                                                          2
fi

#      Initialize      an      array      to      hold      matching      lines.
matches=()
count=0

# Read the log file line-by-line to avoid loading the entire file into memory.
while            IFS=            read            -r            line;            do
  if        [[        "$line"        ==        *"$KEYWORD"*        ]];        then
    ((count++))
    matches+=("$line")
  fi
done                                <                                "$LOG_FILE"

#        Output        the        total        count        of        matches.
printf "Keyword '%s' found %d time(s) in %s\n" "$KEYWORD" "$count" "$LOG_FILE"

#  If  there  are  matches,  display  the  last  five  using  tail  logic.
if          ((          count          >          0          ));          then
  printf       "---       Last       5       matching       entries       ---\n"
  # Use printf to join array elements with newlines, then pipe to tail.
  printf       "%s\n"       "${matches[@]}"       |       tail       -n       5
fi
```

This script is pretty straightforward - it checks a given log file for a specific keyword, counts how many times it shows up, and then displays the last five lines where the keyword was found. So, when you run it, you'll first see a message telling you how many times the keyword appeared in the log file. If the keyword was actually found, it'll then show you the last five lines from the log file that contain the keyword. The output is basically just two parts: the total count of the keyword and then the last five matching lines, which can be really helpful for quickly checking what's going on in a log file.

Script 5: 05-manifesto.sh

```
#!/bin/bash
#                    Author:                    Arjun                    (24BAI10141)
#  ---------------------------------------------------------------------------
#                              Script:                              05-manifesto.sh
# Purpose: Interactively collect user input and generate a personalized Open
# Source manifesto, appending it to a text file named after the current user.
#  ---------------------------------------------------------------------------

# Prompt the user for three pieces of information.
read -p "Enter your favorite FOSS project: " fav_project
read -p "Why do you contribute to open source? " contribution_reason
read -p "What is your vision for the future of open source? " vision

# Construct a paragraph using the collected answers.
MANIFESTO="As a passionate advocate of $fav_project, I believe that
$contribution_reason. My vision for the future of open source is $vision."

# Define the output file name based on the current user.
OUTPUT_FILE="${USER}.txt"

# Append the manifesto to the file, creating it if it does not exist.
printf "%s\n\n" "$MANIFESTO" >> "$OUTPUT_FILE"

# Inform the user of the successful operation.
printf "Your manifesto has been saved to %s\n" "$OUTPUT_FILE"
```

This script basically asks the user for their favorite open source project, why they contribute to open source, and their vision for the future of open source. It then takes this info and creates a paragraph, which is like a personalized manifesto. This manifesto is then saved to a text file named after the current user, and a message is printed to the terminal saying where the file was saved. So, when you run the script, you'll be prompted to enter the required info, and after that, you'll see a message telling you that your manifesto has been saved to a file, and you can go check that file to see your generated manifesto.

Conclusion

In conclusion, Git is a highly capable and flexible version control system that is well-suited to a wide range of projects and users. Its high level of scalability, security, and customizability make it a popular choice for many developers, and its widespread adoption is a testament to its effectiveness. The use of Git has also helped to drive the adoption of other open source software, and has provided a high level of collaboration and community involvement. One of the key benefits of Git is its high level of freedom and flexibility, which allows users and developers to modify and customize the software to suit their needs. The Git community is also highly active and diverse, with many different projects and communities available. The FOSS ecosystem around Git is highly active and diverse, and provides a high level of collaboration, transparency, and accountability. Overall, Git is a highly effective and widely-used version control system that is well-suited to a wide range of projects and users. Its high level of scalability, security, and customizability make it a popular choice for many developers, and its widespread adoption is a testament to its effectiveness. The experience of using Git has been highly positive, with many different benefits and advantages available to users and developers. The use of Git has also helped to drive innovation and improvement, and has provided a high level of freedom and flexibility for users and developers. The conclusion is that Git is a highly capable and flexible version control system that is well-suited to a wide range of projects and users, and its widespread adoption is a testament to its effectiveness.